



ECR (Scan+Lifecycle)+ Jenkins EC2 (SMM)

ECR II

Resources "aws-ecr-repository" "repos" {

AWS يجعل Docker Registry جوا

الهدف من ال ECR بدل ما افزن ال Image بتاعتي

على Docker Hub مخزنها جوا AWS

وده أكثر أمانا وأسرع مع ال EKS

هو بيحل مشكلة لما جيت اسحب ال Image فوارفها

لأنه يكون فاتح على الإنترنت

لكن مع ال ECR كل حاجة جوا AWS

بالتالي Latency أقل و Security أعلى

Authentication

Resources 5 for-each-to-set

فعمل Loop ولخصيت كل ال Repos اللي عايزها

لل Images بتاعتي

ImageTag=IMMUTABLE من أهم الإعدادات

لو عنك Image : V1 بعد اسبوع مثلا حد على Push

لنفس ال Image بنفس ال Version

لو ال Repository Mutable ممكن التغيير ودي كارثة

بالتالي ال Production لازم يكون Version مبيتغيرش

IMMUTABLE يعني ال Image : V1 تترفع مرة واحدة

* Integrity * Rollback * Audit يحافظ على

تقدر ترفع Image عادي V2 لكن لو نفس ال Image

مع نفس ال Version AWS كيرقها

ECR = []

Image Scanning Configuration { Scan-on-Push=true }

كل Image تم رفعها Push الى AWS ECR Scan يفحص على Critical * vulnerabilities من غيره هذا Image فيها ثغرات

أي فرق بينه وبين Trivy

ECR Scanner اداة تشغيل وتشايف ال Images

يعني بعد شهر يقدر انه يكتشف ان في ثغرة في ال Image

Trivy بيتمثل وقت ال Build فقط هو واداة وعشان اكتشف ثغرة جديدة لازم اشغله تاني

الفرق بينهم

ECR Scanner	Trivy	المقارنة
بعد ال Push	قبل ال Push	وقت التشغيل
X	✓	يمنع دفع ال Image
X	✓	يعمل داخل CD/CI
✓	X	مدمج مع AWS
✓	لا، اعادة تشغيل	يكتشف ثغرات جديدة بعد اعادة
X	✓	مناسب لمشاريع ال Pipeline
✓	محدود	مناسب للمراقبة المستمرة

Build

Trivy Scan → Push ECR

ECR Scanner ← يحفظ نتيجة ال Scan

ECR (1)

Encryption-Configuration ← كل Images تتخزن مشفرة

Tags ← AWS Metadata أو بيانات وصفية
بضيفها لأي Resource في AWS
بديل ما يكون عندي ECR اسمها 01234-أ تضيف له
Project=Vprofile

LifeCycle Policy ← عبارة عن قواعد بتقول لـ AWS
لما الـ Image تبين قديمة اذفها تلقائياً

each.value.name ← هات اسم الـ repo الحالي
Policy=JSONCode ← الـ AWS ECR API

بببوقع الـ Policy بمبيغة JSON
لكن Terraform يكتبه بمبيغة HCL
بيحول بيانات الـ HCL إلى JSON تلقائياً

Rules ← بقدر امل اكر من Rules
ولما اضيف Rule بخلها رقم كأولوية في التنفيذ
يعني Rule 1 بخلها 1=rulePriority

Selection={ ← بحد في Image سيطيع عليها القاعدة
من غير الـ Selection حيث تعرف الـ Policy ما الذي سيعمل

CountType="imageCountType" ← اختلف بعدد معين من
الـ Images وانا عدد في الـ Image بحد

action={ ← بعد ان حدونا في Image سيعمل اختيارها
من بعد في العمل فيها أي

action="expire" ← اذف الصور المنتهية
في expire يعني يخلي الـ Image ملتصقة بالذاكرة فتكذف

ملخص ال ECR

الجزء	الهدف	المسألة التي حلها
Repository	مكان تخزين Docker Images	عدم الاعتماد على Docker Hub
for-each	إنشاء عدة Repositories	منع تكرار الكود
IMMUTABLE	منع تغيير نفس ال Tag	الحفاظ على ال Rollback و Audit
Scan on Path	فحص الثغرات الأمنية	منع نشر Images ضعيفة
AES256	تشفير المور المخزنة	حماية البيانات المخزنة
Tags	تنظيم وعادة ال وارج	سهولة البحث والتكلفة والأمن
Lifecycle	حذف المور القديمة تلقائياً	تقليل التكلفة ومنع تكرار ال Image

IAM

(2)

هل ال **EC2** يعتبر **User** ؟ لا
طبع من بقى هياكل المصاحبة

هنا ياتي دور ال **IAM Role**
IAM Role ← مجموعة من المصاحبة ممكن
لكيان يستخدمها مؤقتة لأن ال **Role** من مميزات
Access Key و **Password** هو مزيج **Permission**
لية **Role** علوا ال **Role**

لأن الناس زمان كانوا بيحفظو **(Key+Secret Access)**
في السيرفر ودي كارثة لو دخل السيرفر هلاقي المفاتيح
الحل كان في ال **Role** و **Secret** ال **Role**

Policy ← مستند **JSON** يحدد المسموح والممنوع
aws-iam-role ← **Resource** كذا هتعمل **IAM** كدوره

والهدف منها ان **EC2** **Jenkins** يحصل على صلاحية داخل **Role**
assume-role-policy ← من أهم الأسطر

دا السطر **Policy** **Trust** ← يعني من يقدر يستخدم ال **Role**
وليس هيقدر يحصل في

Version="2012-10-7" ← دي من نسخة **IAM**
دي نسخة لسياسة ال **Policy Language**

كل **Statement** عبارة عن **Rule** مستقلة
ممكن ياتي عندي **Rule1**, **Rule2**

Effect="Allow" ← يعني اسمح وممكن تكون **Deny**

Action="sts:AssumeRole" ← هو يقول المسموح هنا
هو عملية واحدة بس وهنا السطر دا بيمنح ال **EC2**

انها تستخدم ال **AssumeRole** لكن لسا مبدئية
هو هيقدر يحصل في



IAM (2)

Principal = $\rightarrow$ من الذي سيتعامل مع ال Role
 Service = "ec2.amazonaws.com" $\rightarrow$ في EC2 مرتبطة
 بال Role الذي سيتم التعامل مع Assume Role
 وبالتالي يحصل على Credentials مؤقتة من STS

يبقى كذا AWS فاصل بين حاجتين مهمتين
 1 من يستطيع استخدام ال Role $\rightarrow$ Assume Role
 2 ماذا يستطيع ال Role ان يفعل $\rightarrow$ IAM Policy

Role $\rightarrow$ Resource وظيفة يربط Role
 مع Managed Policy موجودة مسبقا في AWS
 لافظ كلمة يربط

هو لا ينشئ Policy جديدة
 ال Managed Policy عبارة عن Permission بدل
 ما تكتب في AWS-Permission بتوفّر Policies جاهزة
 فتتبع بس بتعامل Attach

Role = aws-iam-role-Jenkins $\rightarrow$ يعني اربط ال Policy
 مع ال Role

arn:aws:iam::AmazonSSMManagedInstanceCore $\rightarrow$ يعني
 Policy

AWS هو ال كتابها من ان
 يعني ال EC2 صلاحية يتكلم مع AWS System manager
 وفي Service كخدمة
 Run Command Session Manager
 احنا بنسخدم Session Manager



IAM (2)

Policy_arn = arn:aws:iam::aws:policy/Amazon

يعني Jenkins مدمج على ال ECR
علشان يعمل Push, Pull ويشتغل ال Repository
ليه افترنا BlueUser لأنها تسمح بكل العمليات
على ال Repositories وال Images بدون اعطاء صلاحية ابارية
على IAM أو حساب AWS نفسه

resource = aws:iam::role:Policy

Role Resource بـ Inline Policy و يرتبط بال Role

في AWS عنك نوعين من ال Policies

Managed Policy ← Policy الحاجة من AWS

Inline Policy ← Policy مرتبطة بـ Role

واحدة فقط لو اتعرفت ال Role تتعرف ال Policy

تستخدم managed للحاجة العامة زي S3 Read Only

SSM

تستخدم Inline لما تكون Permission مجموعة

لشيفر واحد يعني Jenkins فقط محتاج يعمل

Push و Pull ولا في Role ثانية محتاجة الـ Action

بالتالي هي محتاجة لـ Jenkins

Action ← يقول لـ AWS تستخدم ال API

DescribeCluster ← دي هي أهم ال Permissions

لما Jenkins يشتغل ← Kubectl get Pods

تحتاج ال AWS CLI بـ aws eks get-token

لما تـ aws eks update-kubeconfig ← name

الـ AWS CLI بـ DescribeCluster الـ API Call

محتاج زي Endpoint بـ API Server, Cluster

وبعض حاجات الـ Kubernetes دي في ملف Kubeconfig و aws

اعل Kubectl get Pods. Kubectl بـ Endpoint من Kubeconfig

• يطلب منك من AWS • يعمل بالـ API بالـ Kubernetes

IAM (2)

aws eks list-clusters

السماح لا role أنه يجب قائمة بكل EKS Clusters الموجودة في AWS Account حسب الـ Region

DescribeCluster ← تعرف تفاصيل Cluster حسب
List-Clusters ← تعرف أسماء جميع الـ Clusters

Action = ["sts: GetCallerIdentity"]

السماح لا Role أو الـ User أنه يستلزم AWS : أن يكون هو من يطلب بيانات عن الهوية الحالية

لوانت على Jenkins وتحتاج
مخرج بيانات ← وينستخدم دي بسم عنوان
في الـ CI/CD كمرجعاً بـضماناً كي وان الـ Credentials
التي ممكن نالها

الجزء	السياسة	السياسة المطلوبة
aws-iam-role-policy	إنشاء Inline Policy	إعطاء صلاحيات لـ Jenkins
role	دعم الـ Policy بالـ Role	تطبيق الـ Role على Jenkins
eks:	عزل الـ Clusters	ممنوع الوصول
eks: DescribeCluster	قراءة بيانات الـ Cluster	تكون الـ update الـ eks ممكن
sts: GetCallerIdentity	معرفة هوية الـ Role	تكون الـ update الـ eks ممكن
↳ Resource	تطبيق السياسة على كل الموارد	

IAM [2]

Resource "aws-iam-instance-profile" ←

دا Resource بيشرح IAM Instance Profile

طيفه هو Instance Profile دا اية ؟

هو عبارة عن wrapper (غلاف) حوالي IAM Role

من بيضيف صلاحية ومن بيخل ال Policy

من بيحل Authentication هو مجوز Container

يخلي ال EC2 يقدر يستفيد ال Role

ال EC2 من يتقدر يمسح ال Role لولده

لزم انا جوال IAM Instance Profile لان ال EC2 من

يتربط بنفسه بال Role من يتربط بنفسه بال Instance Profile

Role = aws-iam-role ← ال IAM Instance Profile

بال IAM Role دي

Security Group [3]

Resource "aws-security-group" ←

ال Security Group جديده

Egress Rule ← عندنا نوعين

Ingress [1] ← من يدخل للسيرفر

Egress [2] ← السيرفر خارج لمن